Verifiable AI via the ℓ^1 Coboundary Norm

Domain Graphs, Hallucination Bounds, and the Autopoietic Verification Layer

Author: Jeremy H. Carroll **Date:** March 2026 **Version:** v1.0 (Pre-Print)

Abstract

Current AI systems large language models, neural networks, diffusion models are trained by minimizing ℓ^2-adjacent loss functions (mean squared error, cross-entropy, KL divergence). ℓ^2-type objectives permit interpolation across inconsistent data, which can contribute to hallucination in the absence of explicit structural constraints, as the mean of conditional distributions on contradictory data may reflect no actual ground truth.

We propose a verification framework based on the ℓ^1 coboundary norm. Given a domain modeled as a finite directed graph $G = (V, E)$ with concepts as vertices and relationships as edges, an AI system's output defines a section $s : V \to F(V)$ assigning claims to concepts. The ℓ^1 coboundary norm

$$I(s) = \sum_{e \in E} \|\delta_e(s)\|$$

measures the total inconsistency of this section across all edges. Within the class of defect measures satisfying locality, additivity, faithfulness, and symmetry, the uniqueness theorems of Papers 000–002 identify this as the unique norm preserving structural topology. The framework's verification guarantee is exactly as strong as the domain graph specification.

The Hodge decomposition of Paper 003 splits any defect field into an exact (repairable) component and an irreducible (cohomological) component. The irreducible component $\Phi([\delta])$ is the domain's intrinsic complexity — the **domain inconsistency floor (Φ)** — the minimum inconsistency that *any* system must exhibit on that domain. The autopoietic iterator of Paper 005 provides a convergent repair algorithm that drives the repairable component to zero.

Together, these results yield: (1) a computable hallucination score for any AI output on any graph-modeled domain; (2) a decomposition into fixable errors and irreducible domain complexity; (3) a self-repairing verification layer with proved convergence; and (4) a categorical framework for transferring verification across domains.

Scope. This paper applies established mathematical results (Papers 000–005) to AI verification. The mathematical foundations are proved; the AI application is a construction [Layer B] whose empirical validation is ongoing.

---

1. Why AI Systems Hallucinate: A Structural Argument

1.1 The ℓ^2 Origin of Hallucination

[Layer A — standard machine learning theory; the mean/median distinction is classical (Huber 1964, Vapnik 1998). The connection between cross-entropy and mean-seeking behavior follows from the information-geometric structure of exponential families (Amari 1985).]

Consider a training set containing two contradictory facts:

- Source A: "The boiling point of water is 100C at sea level."
- Source B: "The boiling point of water is 95C at sea level." (incorrect)

An ℓ^2-type objective minimizes squared deviations, permitting distributions that interpolate between contradictory sources: the model can learn to assign probability mass between points, selecting phantom average values that may appear in *neither* source. This interpolation is a mathematical consequence of how ℓ^2 minimization handles inconsistent data without strict topological constraints.

The ℓ^1 minimizer, by contrast, finds the *median*, reducing linear deviation without compounding across orthogonal directions. It strictly resists interpolating between contradictory boundaries. While real language model training is highly complex (e.g., varying decoding strategies, out-of-distribution shifts, missing constraints), ℓ^2-driven interpolation across inconsistent logic serves as a primary contributing structural root to systemic hallucination.

1.2 The Problem at Scale

This extends beyond individual facts. A large language model trained on billions of documents encounters contradictions at every scale:

- **Factual contradictions**: different sources disagree on dates, numbers, names.
- **Framing contradictions**: the same event described with opposite causal narratives.
- **Ontological contradictions**: different domains use the same terms with incompatible meanings.

The ℓ^2 training objective averages over all of these, producing a model that speaks confidently about a world that is the *mean of all worlds in the training data* — a world that does not exist. The model's internal representation is a superposition of contradictory worldviews, and any particular output is a sample from this superposition. There is no mechanism within the ℓ^2 framework to detect, quantify, or bound this inconsistency.

1.3 Why Post-Hoc Fixes Are Insufficient

Current approaches to hallucination mitigation include:

Approach	What it does	What it cannot do
RAG (Retrieval-Augmented Generation)	Grounds output in retrieved documents	Cannot bound residual inconsistency between retrieved documents and model priors
Confidence calibration	Estimates model uncertainty	LLMs are confidently wrong; calibration requires held-out data from the deployment domain
RLHF	Shapes behavior via reward model	Reward model is itself ℓ^2-trained; shifts the hallucination, doesn't bound it
Constitutional AI	Encodes rules as natural language	Rules are processed by the same ℓ^2-optimized model; no formal guarantee
Fact-checking	Verifies individual claims post-hoc	Does not scale; misses structural inconsistencies between claims

None of these provide a *mathematical bound* on the total inconsistency of the output with respect to a domain. They are engineering patches on a structural problem.

---

2. Domain Graphs and the Coboundary

2.1 Modeling a Domain as a Graph

[Layer B — construction]

Definition 2.1. A *domain graph* is a finite directed graph $G = (V, E)$ where: - **Vertices** V represent concepts, entities, or propositions in a knowledge domain. - **Edges** E represent relationships, implications, or constraints between concepts.

Each vertex v carries a *fiber* $F(v)$ — the space of possible claims about that concept. Each edge $e = (u, v)$ carries a *restriction map* $r_e : F(v) \to F(u)$ — the constraint that the claim at v must be compatible with the claim at u via the relationship e.

Examples:

Domain	Vertices	Edges	Fiber
Medical diagnosis	Symptoms, diseases, treatments	Causal/correlational links	Probability assignments
Legal reasoning	Statutes, precedents, facts	Citations, implications	Interpretations
Scientific knowledge	Hypotheses, experiments, data	Evidential support	Confidence levels

Domain	Vertices	Edges	Fiber
Software architecture	Modules, interfaces, types	Dependencies, type constraints	Implementation states

This is a Banach presheaf over a finite poset — the exact structure analyzed in Papers 000–002.

2.2 Sections and Defects

[Layer A — from Papers 000–002]

Definition 2.2. A *section* s of a domain graph assigns to each vertex v a claim $s(v) \in F(v)$. An AI system's output on a domain defines such a section.

Definition 2.3. The *coboundary defect* at edge $e = (u, v)$ is:

$$\delta_e(s) = r_e(s(v)) - s(u) \in F(u)$$

This measures the inconsistency between what vertex v claims (projected through the constraint e) and what vertex u claims directly. When $\delta_e(s) = 0$, the claims are consistent across edge e. When $\delta_e(s) \neq 0$, there is a contradiction.

Definition 2.4. The *total defect* of a section is the ℓ^1 coboundary norm:

$$I(s) = \sum_{e \in E} \|\delta_e(s)\|$$

2.3 Why ℓ^1 and Only ℓ^1

[Layer A — Theorem 2.1 of Paper 002, Theorem 4.2 of Paper 000]

Theorem 2.5 (ℓ^1 Uniqueness). Any defect measure ν on the cochain space $C^1(G, \mathbb{R}^k)$ satisfying:

1. **Edge locality**: $\nu(\delta) = \sum_e \nu_e(\delta_e)$ — each edge contributes independently.
2. **Coordinate additivity**: defects on disjoint channels aggregate without cross-terms.
3. **Faithfulness**: $\nu_e(\delta_e) = 0 \iff \delta_e = 0$ — no nonzero defect is invisible.
4. **Symmetry**: structurally isomorphic edges are treated identically.

is proportional to the ℓ^1 norm: $\nu(\delta) = \alpha \sum_e \|\delta_e\|_1$ for some $\alpha > 0$.

This uniqueness is formal and bounded firmly within the axiomatic class. The axioms themselves specify the minimal conditions for the existence of consistent local observers (Paper 000, Section 1.2):

- **Locality** means an auditor can check one edge without global access.

- **Additivity** means errors on disconnected components don't cancel.
- **Faithfulness** means no real inconsistency is invisible to the measure.
- **Symmetry** means the measure doesn't depend on arbitrary labeling.

Any organization deploying AI in a domain where wrong answers have consequences implicitly requires all four properties. The theorem says: then you must use ℓ^1.

2.4 What Other Norms Get Wrong

[Layer A — direct consequences of the axioms]

The ℓ^2 failure (cancellation). $\|(+1,-1)\|_2 = \sqrt{2}$, but the ℓ^2 norm of the *sum* permits cancellation: errors of opposite sign partially mask each other. An AI system could have large contradictions in opposite directions that appear small under ℓ^2 measurement. This is precisely the hallucination mechanism: the ℓ^2-trained model has learned to balance contradictions rather than resolve them.

The ℓ^∞ failure (blindness). $\|(1,\varepsilon)\|_\infty = 1$ regardless of ε. The max-norm sees only the worst single edge and ignores all others. A system with one large error and a thousand small errors looks the same as a system with one large error and zero small errors. Distributed inconsistency is invisible.

The ℓ^1 guarantee (no hiding). $\|(+1,-1)\|_1 = 2$. Every nonzero defect contributes. Nothing cancels. Nothing hides. The total defect is the honest sum of all local inconsistencies.

---

3. The Hodge Decomposition: Repairable vs. Irreducible

3.1 Not All Defects Are Equal

[Layer A — Paper 003]

A crucial insight from Paper 003 is that defects decompose into two fundamentally different kinds:

Definition 3.1. The ℓ^1-*native decomposition* of a defect field $\delta \in C^1(G)$ is:

$$\delta = d_0 f^* + r^*$$

where: - $d_0 f^*$ is the **exact (gauge) component** — the part of the defect that can be removed by changing the section s without changing the domain graph. These are *fixable errors*. - r^* is the **irreducible (cohomological) component** — the minimum-cost representative of the cohomology class $[\delta] \in H^1(G)$. These are *structural inconsistencies inherent to the domain*.

The irreducible component has cost:

$$\Phi([\delta]) = \min_{f \in C^0} \|\delta - d_0 f\|_1$$

This is the ℓ^1 quotient norm on $H^1(G)$, computable by linear programming (Paper 003, Section 10).

3.2 What This Means for AI

[Layer B — application]

Given an AI system's output section s on a domain graph G:

1. **Compute** $I(s) = \sum_e \|\delta_e(s)\|$ — the total hallucination score.
2. **Decompose** $\delta(s) = d_0 f^* + r^*$ via LP — separate fixable from irreducible.
3. **Compute** $\Phi([\delta]) = \|r^*\|_1$ — the irreducible domain complexity.
4. **Report**:
   - **Repairable defect**: $I(s) - \Phi([\delta])$ — the amount of inconsistency that *can* be fixed by changing the AI's answers without changing the domain model.
   - **Domain inconsistency floor (Φ)**: $\Phi([\delta])$ — the minimum inconsistency that *any* system must exhibit on this domain, regardless of how good it is.
   - **Hallucination ratio**: $(I(s) - \Phi([\delta]))/I(s)$ — the fraction of the defect that is the AI's fault vs. the domain's inherent complexity.

The irreducible defect Φ is a property of the domain, not the AI. If a medical knowledge graph has contradictory guidelines from different professional bodies, the resulting $H^1 \neq 0$ means *no system* can be fully consistent on that domain. Knowing this is itself valuable: it tells you where domain consensus needs to be built before AI deployment can be reliable.

3.3 The Inflation Ratio: Quantifying ℓ^2 Failure

[Layer A — Paper 003, Section 6]

Paper 003 proves that ℓ^2 Hodge projection generically *inflates* the ℓ^1 cost of the irreducible component. On cycle graphs C_n, the inflation ratio is exactly $3 - 2/n$.

This means: if you use standard ℓ^2 methods (least-squares regression, neural network training, PCA-based dimensionality reduction) to estimate the irreducible defect, you will *overestimate* it by a factor approaching 3. You will attribute to "inherent domain complexity" what is actually fixable error — because ℓ^2 spreads mass where ℓ^1 concentrates it. This has direct consequences for AI deployment: ℓ^2-based uncertainty estimates systematically overstate irreducible uncertainty and understate the AI system's correctable errors.

3.4 Worked Numerical Example: Medical Diagnostic Domain

[Layer B — pedagogical construction demonstrating the framework end-to-end with concrete numbers]

To make the framework concrete, we present a complete worked example on a small medical diagnostic domain.

Step 1: Domain Graph Construction

Consider a simplified diagnostic domain with 5 vertices (concepts) and 8 directed edges (constraints):

Vertices $V = \{\text{Fever}, \text{Headache}, \text{Flu}, \text{Migraine}, \text{Dehydration}\}$

Fibers: Each vertex carries a probability fiber $F(v) = [0, 1]$ representing the probability that a patient has that symptom or condition.

Edges (with restriction maps): - e_1: Flu $\to$ Fever — if Flu prob $= p$, then Fever prob $\geq 0.8p$ (flu typically causes fever) - e_2: Flu $\to$ Headache — if Flu prob $= p$, then Headache prob $\geq 0.6p$ - e_3: Migraine $\to$ Headache — if Migraine prob $= p$, then Headache prob $\geq 0.9p$ - e_4: Dehydration $\to$ Headache — if Dehydration prob $= p$, then Headache prob $\geq 0.5p$ - e_5: Dehydration $\to$ Fever — if Dehydration prob $= p$, then Fever prob $\geq 0.3p$ - e_6: Fever $\leftrightarrow$ Dehydration — mutual exclusion constraint: $|\text{Fever prob} - \text{Dehydration prob}| \geq 0.4$ (for simplicity in this example, we model a scenario where high fever and high dehydration probabilities should not both be elevated simultaneously without additional context) - e_7: Headache $\to$ Migraine — if Headache prob > 0.7 and no other causes, then Migraine prob $\geq 0.5 \times$ Headache prob - e_8: Flu $\leftrightarrow$ Migraine — Flu and Migraine are competing diagnoses: if Flu prob > 0.5, then Migraine prob < 0.3 (typically, having flu reduces the likelihood of migraine being the primary diagnosis)

Simplified representation: To make the defect computation transparent, we represent each edge constraint as a linear inequality that should be satisfied for consistency. The defect at edge e is the violation magnitude when the constraint is not met.

Step 2: AI System Output (Initial Section)

An AI diagnostic system evaluates a patient and produces the following probability assignments:

```
s(Fever) = 0.7
s(Headache) = 0.8
s(Flu) = 0.6
s(Migraine) = 0.5
s(Dehydration) = 0.4
```

Step 3: Coboundary Computation

We compute the defect $\delta_e(s)$ on each edge by checking constraint violations:

Edge	Constraint	AI Output	Defect $\|\delta_e(s)\|$
e_1	Fever $\geq 0.8\times$ Flu	$0.7 \geq 0.8 \times 0.6 = 0.48$	0 (satisfied)
e_2	Headache $\geq 0.6\times$ Flu	$0.8 \geq 0.6 \times 0.6 = 0.36$	0 (satisfied)
e_3	Headache $\geq 0.9\times$ Migraine	$0.8 \geq 0.9 \times 0.5 = 0.45$	0 (satisfied)
e_4	Headache $\geq 0.5\times$ Dehydration	$0.8 \geq 0.5 \times 0.4 = 0.20$	0 (satisfied)
e_5	Fever $\geq 0.3\times$ Dehydration	$0.7 \geq 0.3 \times 0.4 = 0.12$	0 (satisfied)
e_6	$\|$Fever $-$ Dehydration$\| \geq 0.4$	$\|0.7 - 0.4\| = 0.3 < 0.4$	$0.4 - 0.3 = 0.1$
e_7	If Headache > 0.7 then Migraine $\geq 0.5\times$ Headache	$0.5 \geq 0.5 \times 0.8 = 0.4$	0 (satisfied)
e_8	If Flu > 0.5 then Migraine < 0.3	Flu $= 0.6$, Migraine $= 0.5 \not< 0.3$	$0.5 - 0.3 = 0.2$

Step 4: Total Defect

The total ℓ^1 coboundary defect is:

$$I(s) = \sum_{e=1}^{8} |\delta_e(s)| = 0 + 0 + 0 + 0 + 0 + 0.1 + 0 + 0.2 = 0.3$$

The AI system has a hallucination score of $I = 0.3$ on this domain.

Step 5: Hodge Decomposition (LP Formulation)

To separate repairable from irreducible defect, we solve:

$$\Phi([\delta]) = \min_{f \in C^0(V,\mathbb{R})} \sum_{e \in E} |\delta_e(s) - d_0 f(e)|$$

where $d_0 f(e) = f(v) - f(u)$ for edge $e = (u, v)$, and $f : V \to \mathbb{R}$ represents a gauge transformation (adjustment to the section).

This is a linear program with 5 variables (one per vertex) and constraints for each edge. The LP finds the optimal gauge shift that minimizes residual defect.

Solving the LP (simplified for exposition):

The two violated constraints are: - e_6: Fever-Dehydration separation (defect 0.1) - e_8: Flu-Migraine exclusion (defect 0.2)

By adjusting f(Migraine) down by 0.2, we can eliminate the e_8 defect (bringing Migraine from 0.5 to 0.3). By adjusting f(Dehydration) down by 0.1, we can eliminate the e_6 defect (increasing the separation). These are independent repairs with no topological obstruction.

Result: $\Phi([\delta]) = 0$ (the defects are fully repairable; there is no irreducible cohomological component for this particular configuration).

Step 6: Defect Decomposition

- **Total defect**: $I(s) = 0.3$
- **Irreducible defect**: $\Phi = 0$
- **Repairable defect**: $R = I - \Phi = 0.3$
- **Hallucination ratio**: $\eta = R/I = 1.0$ (100% of the defect is the AI's fault, 0% is irreducible domain complexity)

Step 7: AGI Repair

The Autopoietic Gauge Iterator repairs the section by iteratively applying ℓ^1-median updates:

Iteration 1: Target vertex v^* = Migraine (highest local defect contribution from e_8)

- Neighbors contributing to Migraine: Headache (via e_7), Flu (via e_8)
- Projected values: Headache suggests Migraine ≈ 0.4 (from e_7), Flu constraint suggests Migraine < 0.3 (from e_8)
- Weighted ℓ^1 median: s(Migraine) $\leftarrow 0.3$ (reduced from 0.5)

After Iteration 1: - e_8 defect: Migraine now 0.3, exactly at the boundary — defect reduced to 0 - New total defect: $I(s) = 0.1$ (only e_6 remains)

Iteration 2: Target vertex v^* = Dehydration (only remaining defect at e_6)

- Adjusting Dehydration down to 0.3 or Fever up to 0.8 would satisfy e_6
- Given other constraints (Fever is constrained by e_1, e_5), the median-based repair adjusts Dehydration $\leftarrow 0.3$

After Iteration 2: - e_6 defect: $|0.7 - 0.3| = 0.4$, exactly at the boundary — defect = 0 - New total defect: $I(s^*) = 0$

Convergence: The AGI has repaired the section to full consistency in 2 iterations.

Step 8: Hallucination Certificate

The final certificate for this AI system's output on this domain graph is:

$$\mathcal{H}(\mathcal{A}, G) = (I = 0.3, \Phi = 0, R = 0.3, \eta = 1.0, \mathcal{C} = \emptyset)$$

Interpretation: - $I = 0.3$: The AI's initial output had measurable structural inconsistency. - $\Phi = 0$: None of this inconsistency is irreducible; the domain graph itself is fully consistent. - $R = 0.3$: All 0.3 units of defect are repairable errors made by the AI system. - $\eta = 1.0$: 100% of the defect is the AI's fault. The domain has no contradictions. - $\mathcal{C} = \emptyset$: No cohomology classes carry irreducible defect (the domain has $H^1(G) = 0$ or a trivial cohomological contribution).

After AGI repair, the certificate for the repaired section s^* is:

$$\mathcal{H}(\mathcal{A}_{\text{repaired}}, G) = (I = 0, \Phi = 0, R = 0, \eta = 0, \mathcal{C} = \emptyset)$$

The repaired output is fully consistent with the domain graph.

What This Example Demonstrates:

1. **Concrete computation**: Every step involves actual numbers that can be verified by hand or with a standard LP solver.
2. **Transparency**: The defects at e_6 and e_8 are explicit violations of domain constraints, not statistical artifacts.
3. **Repairability analysis**: The LP demonstrates that both defects are gauge-removable (no topological obstruction).
4. **Convergent repair**: The AGI eliminates both defects in 2 iterations using ℓ^1-median updates.
5. **Certification**: The final certificate provides a complete, auditable summary of the AI's consistency with the domain.

Contrast with ℓ^2 approach: An ℓ^2-trained system minimizing mean squared error would produce probability assignments that average over the conflicting constraints at e_6 and e_8, potentially outputting Migraine ≈ 0.4 (the mean of 0.5 and 0.3) rather than resolving the conflict. The ℓ^2 defect $\|(0.1, 0.2)\|_2 = \sqrt{0.05} \approx 0.22$ would appear smaller than the ℓ^1 defect 0.3, masking the actual inconsistency. The ℓ^1 framework forces the system to address each edge violation explicitly, without averaging.

This example captures the full pipeline from domain graph construction through defect measurement, Hodge decomposition, repair, and certification — with every number explicit and verifiable. It serves as a template for applying the framework to real-world domains.

3.5 Worked Example: Irreducible Defect ($\Phi > 0$)

We now present a minimal domain where inconsistency is **structural and unavoidable**.

Step 1: Domain Graph Let $V = \{A, B, C\}$ Edges: * $e_1 : A = B$ * $e_2 : B = C$ * $e_3 : A \neq C$

Step 2: Interpretation Constraints imply $A = B$ and $B = C \implies A = C$. But we also require $A \neq C$. This creates a **cycle inconsistency**.

Step 3: Section (AI Output) Let $s(A) = 1, \quad s(B) = 1, \quad s(C) = 1$

Step 4: Coboundary Compute defects: * $e_1 : A = B \implies 0$ * $e_2 : B = C \implies 0$ * $e_3 : A \neq C \implies 1$

Step 5: Total Defect

$$I(s) = 1$$

Step 6: Minimize Defect Try all consistent assignments: * **Case 1:** $A = B = C$ * equality constraints satisfied * inequality violated $\implies$ defect $= 1$ * **Case 2:** break equality (e.g. $A \neq C$) * at least one of $A = B$ or $B = C$ must fail $\implies$ defect ≥ 1

Conclusion:

$$\Phi = 1$$

Step 7: Decomposition

$$R = I - \Phi = 1 - 1 = 0$$

Step 8: Certificate

$$\mathcal{H} = (I = 1, \Phi = 1, R = 0, \eta = 0, \mathcal{C} = \{\text{cycle}\})$$

Step 9: AGI Behavior The Autopoietic Gauge Iterator attempts median updates and evaluates local consistency. But every configuration has defect ≥ 1. So no update reduces I, and the system converges immediately to:

Final State:

$$s^* = s, \quad I(s^*) = \Phi = 1$$

Interpretation (strict, no philosophy) * The inconsistency is **not repairable**. * It is a **cohomological obstruction**. * It resides in a nontrivial cycle in $H^1(G)$.

The verification layer correctly identifies that no section can satisfy all constraints simultaneously and therefore refuses to produce a zero-defect output.

3.6 Cohomological Interpretation of Irreducible Defect

We now connect the irreducible defect in Example 3.5 to nontrivial cohomology.

Proposition 3.6 (Irreducible defect $\iff$ nontrivial cohomology class) Let $\delta \in C^1(G)$ be the coboundary defect of a section s. Then:

$$\Phi([\delta]) > 0 \quad \iff \quad [\delta] \neq 0 \in H^1(G)$$

Proof Sketch: We argue directly from the ℓ^1 formulation.

($\implies$) If $\Phi > 0$, then $[\delta] \neq 0$ By definition: $\Phi = \min_f \|\delta - d_0 f\|_1$. If $[\delta] = 0$, then there exists f such that $\delta = d_0 f$, which implies $\|\delta - d_0 f\|_1 = 0$, contradicting $\Phi > 0$. Therefore, $\Phi > 0 \implies [\delta] \neq 0$.

($\impliedby$) If $[\delta] \neq 0$, then $\Phi > 0$ If $[\delta] \neq 0$, then $\delta \notin \mathrm{im}(d_0)$. So for all f, $\delta - d_0 f \neq 0$. By faithfulness of the ℓ^1 norm, $\|\delta - d_0 f\|_1 > 0$. Taking the minimum over all f, $\Phi > 0$.

Conclusion:

$$\Phi = 0 \quad \iff \quad [\delta] = 0$$

$$\Phi > 0 \quad \iff \quad [\delta] \neq 0$$

■

Irreducible defect is exactly the ℓ^1 norm of a nontrivial cohomology class; it vanishes if and only if the defect is globally exact.

In Example 3.5, the cycle $A \to B \to C \to A$ carries a nontrivial cohomology class, and the inequality constraint forces a nonzero representative, yielding $\Phi = 1$.

Examples 3.4 and 3.5 demonstrate the two pure, isolated regimes of the verification framework: complete repairability ($\Phi = 0$) and topologically protected inconsistency ($\Phi > 0$), corresponding to trivial and nontrivial cohomology respectively. In practice, real-world knowledge domains are richly mixed, exhibiting clusters of cleanly repairable logic entangled with dense, cyclic ambiguities that yield nonzero cohomological floors ($\Phi > 0$).

---

4. The Autopoietic Iterator: Self-Repairing AI

4.1 The Algorithm

[Layer A — Paper 005, with convergence proof in companion document]

The Autopoietic Gauge Iterator (AGI) is a repair algorithm that drives the repairable component of the defect to zero:

```
Input: Domain graph G = (V, E), section s, tolerance epsilon
Output: Repaired section s* with I(s*) <= Phi([delta]) + epsilon

Repeat:
    1. DETECT: Find the vertex v* with maximum local defect
```

```
      C_v(s) = sum_{e incident to v} ||delta_e(s)||

   2. OPTIMIZE: Replace s(v*) with the weighted median of
      its neighbors' projected values
      s(v*) <- median_L1({r_e(s(u)) : u ~ v*})

      (The weighted median is defined with respect to edge weights induced by the $\ell^1$

   3. SUBDIVIDE: If the residual defect at v* exceeds a
      threshold after optimization, refine the domain graph
      locally (add vertices and edges to resolve the
      topological obstruction)

Until I(s) - Phi([delta]) < epsilon
```

The median update is a local realization of an underlying ℓ^1 primal–dual optimization flow (Paper 005, Section 8), ensuring convergence to a global ℓ^1-minimal configuration.

4.2 Convergence Guarantees

[Layer A — proved in agi_convergence_proof.md]

Theorem 4.1 (Monotone Descent). Each iteration of the AGI strictly decreases the total defect:

$$I(s) - I(s') \geq \frac{\deg(v^*)}{2} |s(v^*) - m^*|$$

where m^* is the weighted median at vertex v^*.

Theorem 4.2 (Convergence Rate). The AGI reaches an ε-approximate fixed point in at most $O(I_0/\varepsilon)$ iterations, where I_0 is the initial defect. Under a non-degeneracy condition, convergence is geometric: $O(\log(1/\varepsilon))$ iterations.

Theorem 4.3 (Cohomological Floor). The AGI converges to $I^* = \Phi([\delta])$ — the irreducible cohomological defect. It removes all repairable error and stops at the topological boundary.

4.3 What This Means for AI

[Layer B — application]

The AGI provides a *post-processing verification and repair layer* for any AI system:

1. The AI generates output (section s on domain graph G).
2. The AGI computes $I(s)$ and decomposes the defect.

3. For each repairable inconsistency, the AGI proposes a median-based correction.
4. The repaired output s^* has defect $I^* = \Phi([\delta])$ — the minimum achievable.
5. If $\Phi > 0$, the AGI reports which cycles in G carry irreducible inconsistency, identifying the specific domain contradictions that prevent full consistency.

This is not fine-tuning. It is not RLHF. It is a *mathematical verification layer* that sits between the AI and the user, acting structurally as a **consistency projection operator**—not a truth optimizer—with proved convergence and a computable bound on residual error.

Proposition 4.X (AGI realizes the ℓ^1 projection). The Autopoietic Gauge Iterator converges to a section s^* satisfying:

$$I(s^*) = \Phi([\delta(s)])$$

and therefore computes a minimizer of the ℓ^1 coboundary norm over the gauge orbit of s. In particular, the repairable defect $R(s) = I(s) - \Phi([\delta])$ is exactly the portion eliminated by the iterator.

---

5. The Hallucination Certificate

5.1 Definition

[Layer B — construction]

Definition 5.1. A *hallucination certificate* for an AI system $\mathcal{A}$ on a domain graph G is a tuple:

$$\mathcal{H}(\mathcal{A}, G) = (I, \Phi, R, \eta, \mathcal{C})$$

where: - $I = I(s_{\mathcal{A}})$ is the total defect of the AI's output section. - $\Phi = \Phi([\delta])$ is the irreducible domain complexity. - $R = I - \Phi$ is the repairable defect (the AI's fault). - $\eta = R/I$ is the hallucination ratio ($0 \leq \eta \leq 1$). - $\mathcal{C} \subset H^1(G)$ is the set of cohomology classes carrying irreducible defect, identifying *which domain contradictions* prevent full consistency.

5.2 Properties

[Layer A — follows from Papers 002–003]

The hallucination certificate has the following guaranteed properties:

1. **Completeness**: $I = 0 \iff$ the AI's output is fully consistent with the domain graph. No inconsistency is hidden.
2. **Soundness**: $\Phi > 0 \implies$ no system can achieve $I = 0$ on this domain. The certificate correctly identifies irreducible complexity.

3. **Computability**: I is computed in $O(|E|)$ time. Φ is computed by linear programming in polynomial time.
4. **Compositionality**: For domain graphs G_1, G_2 with no shared edges, $I(s|_{G_1 \sqcup G_2}) = I(s|_{G_1}) + I(s|_{G_2})$. The certificate decomposes over independent subdomains.
5. **Monotonicity under repair**: After AGI repair, $I(s^*) \leq I(s)$ and $R(s^*) = 0$.

5.3 What You Can Put in a Contract

[Layer B — application]

The hallucination certificate enables contractual guarantees of a form that no current AI system can provide:

> *"On domain G (specified as a graph with $|V|$ concepts and $|E|$ relationships), the AI system's output has total defect $I \leq X$, of which at most $\Phi = Y$ is irreducible domain complexity. The system's repairable defect is $R = X - Y$, and after verification-layer repair, the residual defect is $\Phi = Y$. The specific domain contradictions preventing full consistency are identified in the cohomology report $\mathcal{C}$."*

This is a number. It goes in a contract. It is auditable. It is reproducible. No current AI vendor can provide anything comparable.

The hallucination certificate guarantees bounded structural inconsistency with respect to the specified domain graph; it does not guarantee factual correctness beyond the fidelity of the domain model.

---

6. The Categorical Framework: Cross-Domain Transfer

6.1 Domains as Objects in a Category

[Layer B — from Paper 006]

Definition 6.1. The *category of domain graphs* **Dom** has: - **Objects**: Domain graphs (G, F, r) — graph, fibers, restriction maps. - **Morphisms**: Graph homomorphisms $\phi : G_1 \to G_2$ that commute with restriction maps: $r_{\phi(e)} \circ \phi_* = \phi_* \circ r_e$.

A morphism $\phi : G_1 \to G_2$ is a *refinement* if it is injective on vertices (the target domain has at least as much structure as the source).

Theorem 6.2 (Defect Functoriality). [Layer A — from Paper 000, Axiom 3] For any domain morphism $\phi : G_1 \to G_2$ and section s on G_2:

$$I(\phi^* s) \leq I(s)$$

Pulling back to a coarser domain cannot increase defect. Equivalently: measuring on a finer domain graph can only *reveal* more inconsistency, never less.

6.2 Practical Consequence

[Layer B — application]

This means: - If you verify an AI system on a detailed domain graph and it passes, it will also pass on any coarser version of that domain. - If an AI system fails on a coarse domain graph, no amount of additional detail will fix it — the inconsistency is already visible at the coarse level. - Domain graph refinement is a *monotone information operator*: more structure means more potential defect to detect.

This provides a principled approach to domain graph construction: start coarse, compute the hallucination certificate, refine where the defect concentrates, repeat. The autopoietic subdivision step (Paper 005) automates this process.

---

7. Architecture: The ℓ^1 Verification Stack

7.1 System Design

[Layer B — engineering construction]

```
                    +------------------+
                    |   User / Client  |
                    +--------+---------+
                             |
                    +--------v---------+
                    | Hallucination    |
                    | Certificate      |  <-- Output: (I, Phi, R, eta, C)
                    +--------+---------+
                             |
                +------------v-----------+
                |   Autopoietic Repair   |
                |   Layer (AGI)          |  <-- Proved convergence
                +------------+-----------+
                             |
                +------------v-----------+
                |   Hodge Decomposition  |
                |   (LP Solver)          |  <-- Exact + Irreducible split
                +------------+-----------+
                             |
                +------------v-----------+
                |   Coboundary Computer  |
                |   I(s) = Sum ||d_e||   |  <-- O(|E|) computation
```

```
            +------------+-----------+
                         |
         +--------------v--------------+
         |                             |
+--------v--------+          +--------v--------+
| AI System Output |          | Domain Graph    |
| (section s)      |          | (G, F, r)       |
+------------------+          +-----------------+
```

7.2 Computational Complexity

[Layer A — standard complexity results]

Operation	Complexity	Method
Coboundary computation $\delta_e(s)$	$O(\|E\|)$	Direct evaluation
Total defect $I(s)$	$O(\|E\|)$	Sum of norms
Hodge decomposition	$O(\|E\|^{1.5})$	LP (network simplex)
Single AGI repair step	$O(\Delta_{\max})$	Weighted median
Full AGI convergence	$O(\|V\| \cdot I_0/\varepsilon)$	Iterative
Cohomology computation $H^1(G)$	$O(\|E\| - \|V\| + c)$	Cycle space basis

For a domain graph with 10,000 concepts and 50,000 relationships, the full verification pipeline (coboundary + Hodge + repair + certificate) runs in seconds on commodity hardware. This is not a theoretical exercise — it is deployable infrastructure.

7.3 Integration Modes

[Layer B — engineering]

The ℓ^1 verification layer can be deployed in three modes:

Mode 1: Post-hoc audit. Run the verification pipeline on completed AI outputs. Produces a hallucination certificate without modifying the output. Useful for compliance, auditing, and domain qualification.

Mode 2: Repair loop. Run the AGI on AI outputs to produce corrected versions with minimum defect. The repaired output satisfies $I(s^*) = \Phi([\delta])$. Useful for high-stakes deployments where consistency is required.

Mode 3: Training signal. Use the coboundary defect $I(s)$ as a training loss or reward signal. Replace ℓ^2 cross-entropy with ℓ^1 coboundary loss on a domain graph. Train the AI to minimize structural inconsistency directly. [Layer C — conjectural; requires empirical validation of training dynamics.]

Mode 4: Constraint-guided generation. An extension is to integrate local defect gradients into the generation process, enabling consistency-aware decoding rather than post-hoc repair.

---

8. What This Framework Does Not Solve

[Layer A — honest scope limitation]

8.1 Limitations

1. **Domain graph construction is non-trivial.** The framework assumes a domain graph G is given. Constructing G for a real domain (medicine, law, finance) requires domain expertise and is itself a significant engineering task. The framework does not automate this step — though the autopoietic subdivision (Paper 005) provides a principled refinement procedure once an initial graph exists.

2. **Fiber choice matters.** The fibers $F(v)$ and restriction maps r_e must be chosen to reflect the actual constraints of the domain. Poor choices produce vacuous certificates (everything passes) or impossible standards (nothing passes). Domain modeling is an art, not a theorem.

3. **The framework measures consistency, not truth.** A section can be fully consistent ($I = 0$) and still be *wrong* — it just needs to be internally coherent. If the domain graph itself contains false relationships, the verification layer will enforce consistency with those false relationships. The AGI repair operator enforces consistency, not correctness, and may distort the truth to achieve mathematical coherence. The verification guarantee is only as strong as the domain graph specification. The ℓ^1 framework guarantees *structural integrity*, not *factual correctness*.

4. **Computational cost scales with domain graph size.** For domains with millions of concepts and relationships, the LP solver for the Hodge decomposition becomes the bottleneck. Approximation algorithms exist but weaken the guarantees.

5. **The autopoietic repair changes the output.** Mode 2 (repair loop) modifies the AI's answers to achieve consistency. This may change correct answers to achieve global consistency — the median-based repair does not know which individual claims are true, only which configurations are globally consistent. Human review of repairs is advisable for high-stakes domains.

6. **Mode 3 (training signal) is unproven.** Using ℓ^1 coboundary loss for neural network training is conjectural. The loss landscape, gradient properties (subgradients at non-differentiable points), and interaction

with backpropagation require empirical investigation. This is a research direction, not a deployed capability.

8.2 What Is Not Claimed

This paper does not claim: - That ℓ^1 verification eliminates all AI errors. It bounds *structural inconsistency*, not *factual error*. - That the domain graph is easy to build. It is a prerequisite, not a byproduct. - That the framework replaces human judgment. It provides *quantitative input* to human judgment. - That LLMs are obsolete. The framework is a *verification layer*, not a replacement for generation.

---

9. Comparison with Existing Approaches

9.1 Formal Verification (Software)

Traditional formal verification (model checking, theorem proving, abstract interpretation) proves that a *program* satisfies a *specification*. The ℓ^1 framework proves that an *output* is *consistent with a domain graph*. The key differences:

- Formal verification is binary (pass/fail). The ℓ^1 certificate is quantitative (a defect score with decomposition).
- Formal verification requires a formal specification. The ℓ^1 framework requires a domain graph, which can be built incrementally and refined.
- Formal verification scales poorly to neural networks. The ℓ^1 framework operates on the *output*, not the model, so it is model-agnostic.

9.2 Conformal Prediction

Conformal prediction provides distribution-free confidence intervals for point predictions. It requires exchangeability assumptions and held-out calibration data. The ℓ^1 framework:

- Does not require held-out data (it operates on the domain graph structure).
- Does not require exchangeability (it works on any graph).
- Measures *structural consistency*, not *statistical confidence*.
- Is deterministic, not probabilistic.

9.3 Knowledge Graph Embeddings

Knowledge graph embedding methods (TransE, DistMult, ComplEx) learn continuous representations of entities and relations. The ℓ^1 framework:

- Does not learn representations (it operates on explicit claims).
- Provides exact decomposition (not approximate embedding).
- Has proved convergence for repair (not just training convergence).
- Produces auditable certificates (not opaque embeddings).

9.4 Hallucination Detection Literature

[Layer A — recent ML research; positioning the ℓ^1 framework against statistical approaches]

Recent work on LLM hallucination detection has produced several notable approaches. We position the ℓ^1 coboundary framework against these methods:

SelfCheckGPT (Manakul et al., 2023) **Approach**: Samples multiple outputs from the LLM for the same prompt and measures consistency across samples. High variance indicates hallucination.

Comparison with ℓ^1 framework: - **SelfCheckGPT**: Statistical consistency via repeated sampling (stochastic). - ℓ^1 **framework**: Structural consistency via domain graph constraints (deterministic). - **Key difference**: SelfCheckGPT detects *stochastic inconsistency* (the model gives different answers on repeated trials). The ℓ^1 framework detects *structural inconsistency* (a single output violates domain relationships). A model can be self-consistent (low SelfCheckGPT score) but structurally inconsistent (high ℓ^1 defect) if it confidently produces the same wrong answer repeatedly. - **Complementarity**: SelfCheckGPT catches sampling noise; ℓ^1 catches systematic structural errors.

HaluEval (Li et al., 2023) **Approach**: Constructs a benchmark with human-annotated hallucinations across multiple tasks (QA, summarization, dialogue). Uses LLM-based evaluators to detect hallucinations.

Comparison with ℓ^1 framework: - **HaluEval**: Benchmark for evaluating hallucination detectors; uses LLM-as-judge. - ℓ^1 **framework**: Provides a mathematical measure independent of human annotation or another LLM. - **Key difference**: HaluEval requires ground-truth labels and an evaluator model. The ℓ^1 framework requires only a domain graph and produces a computable score without labels or additional models. - **Advantage of ℓ^1**: No dependence on another potentially-hallucinating LLM or expensive human annotation for every deployment domain. - **Advantage of HaluEval**: Captures semantic hallucinations that may not violate explicit graph constraints.

FActScore (Min et al., 2023) **Approach**: Decomposes a generation into atomic facts and verifies each against a knowledge source (e.g., Wikipedia). The FActScore is the fraction of facts supported by the source.

Comparison with ℓ^1 framework: - **FActScore**: Atomic fact verification against a reference corpus. - ℓ^1 **framework**: Structural consistency verification against a domain graph. - **Similarity**: Both decompose outputs into checkable units (facts vs. section values on vertices). - **Key difference**: FActScore checks *factual correctness* (is this fact in the corpus?). The ℓ^1 framework checks *structural consistency* (do these claims satisfy domain constraints?). A fact can be in Wikipedia (high FActScore) but still violate logical relationships with other facts (high ℓ^1 defect). - **Complementarity**: FActScore verifies ground truth;

ℓ^1 verifies coherence. Ideally, both are used: FActScore ensures each claim is grounded, ℓ^1 ensures the claims don't contradict each other.

Retrieval-Augmented Generation (RAG) **Approach**: Ground generation in retrieved documents from an external corpus.

Comparison with ℓ^1 framework: - **RAG**: Shifts the distribution toward a reference corpus. - **ℓ^1 framework**: Verifies consistency regardless of retrieval. - **Key difference**: RAG does not *bound* hallucination — retrieved documents can contradict each other, and the model can still hallucinate when interpolating between them. The ℓ^1 framework *quantifies and bounds* residual inconsistency after RAG. - **Integration**: The ℓ^1 verification layer can be applied to RAG outputs to detect when retrieved documents contradict the domain graph or each other.

Uncertainty Quantification via Conformal Prediction **Approach**: Constructs prediction sets with statistical coverage guarantees based on held-out calibration data.

Comparison with ℓ^1 framework: - **Conformal prediction**: Statistical uncertainty quantification (prediction sets). - **ℓ^1 framework**: Deterministic inconsistency quantification (defect scores). - **Key difference**: Conformal methods provide *probabilistic guarantees* (with probability $1 - \alpha$, the true answer is in the prediction set). The ℓ^1 framework provides *deterministic guarantees* (the total structural inconsistency is at most I). Conformal prediction requires exchangeability assumptions and calibration data from the deployment distribution; ℓ^1 verification requires only a domain graph. - **Trade-off**: Conformal prediction handles statistical variability; ℓ^1 handles structural inconsistency. Both are needed for different aspects of reliability.

Consistency-Based Approaches (Numerous works 2023-2026) Several recent papers measure internal consistency (e.g., checking if $P(\text{answer}|\text{question})$ matches $P(\text{question}|\text{answer})$, or measuring agreement across paraphrased prompts).

Comparison with ℓ^1 framework: - **Consistency metrics**: Measure alignment between different conditional distributions or prompt perturbations. - **ℓ^1 framework**: Measures alignment with domain graph constraints. - **Key difference**: Existing consistency metrics are *model-internal* (do the model's own predictions agree with each other?). The ℓ^1 framework is *model-external* (do the model's predictions agree with domain structure?). A model can be internally consistent but externally inconsistent with the domain. - **Advantage of ℓ^1**: Provides an external reference (the domain graph) rather than relying solely on the model's own beliefs.

Summary Table: Positioning the ℓ^1 Framework

Approach	What it measures	Requires	Guarantees	ℓ^1 framework
SelfCheckGPT	Stochastic consistency	Multiple samples	None (heuristic)	Structural consistency, deterministic
HaluEval	Semantic correctness	Human labels, LLM judge	None (benchmark only)	Mathematical defect score
FActScore	Factual grounding	Reference corpus	None (fraction score)	Structural coherence
RAG	Retrieval-based grounding	External corpus	None (shifts distribution)	Quantified residual inconsistency
Conformal prediction	Statistical coverage	Calibration data, exchangeability	Probabilistic ($1-\alpha$ coverage)	Deterministic defect bound
Internal consistency	Model self-agreement	Multiple queries	None (correlation)	External domain alignment
ℓ^1 **coboundary**	Structural inconsistency	Domain graph	Exact decomposition ($I = \Phi + R$)	Novel contribution

The ℓ^1 coboundary framework occupies a distinct position: it is the only approach that provides a *deterministic, mathematically-proved decomposition* of hallucination into repairable vs. irreducible components, with a *convergent repair algorithm* and a *computable certificate*. It is model-agnostic (works on any AI system's output), requires no calibration data or ground-truth labels, and provides guarantees that are *structural* rather than *statistical*.

Complementarity, not competition: The ℓ^1 framework is complementary to statistical approaches. Ideally, a deployed AI system would use: - **FActScore or RAG** to ground individual facts. - **ℓ^1 coboundary verification** to ensure structural consistency across facts. - **Conformal prediction** to quantify statistical uncertainty. - **SelfCheckGPT** to detect stochastic inconsistency.

Each addresses a different facet of reliability. The ℓ^1 framework's unique contribution is the *structural consistency layer* with proved convergence and decomposition theorems.

---

10. The Observer-Theoretic Interpretation

10.1 AI Systems as Observers

[Layer B — from Paper 000, Section 1.2]

Paper 000 establishes that the four axioms (locality, additivity, faithfulness, symmetry) are the minimal conditions for the existence of *consistent local observers* — agents that can detect, aggregate, and repair local inconsistencies.

An AI system deployed in a domain *is* such an observer. It inspects local relationships (edges), makes claims (section values), and must remain globally consistent. The ℓ^1 coboundary norm is not an arbitrary quality metric imposed from outside — it is the *unique measure that the AI system itself would need to use* to maintain internal consistency, if it were capable of self-audit.

The autopoietic verification layer provides this self-audit capability externally. It performs the consistency check that the AI system's ℓ^2-trained internals cannot perform.

10.2 The Dimension Gap Heuristic

[Layer C — conjectural; implemented in tower L040 but not rigorously proved]

An AI system with training manifold dimension d_{train} and input manifold dimension d_{input} is expected to hallucinate when $d_{\text{input}} > d_{\text{train}}$ — when the deployment domain is richer than the training domain. The dimension gap $d_{\text{input}} - d_{\text{train}}$ is a heuristic measure of the *structural capacity for hallucination*.

The ℓ^1 coboundary norm can detect this gap operationally: a system deployed outside its training manifold should exhibit elevated $I(s)$ on domain graphs that probe the gap dimensions. The hallucination certificate would then identify *which* dimensions of the domain graph are causing elevated defect, providing targeted information about where the AI system's knowledge is insufficient. This connection between manifold dimension gaps and coboundary defect elevation requires formal proof.

10.3 Metacognition as ℓ^1 Inconsistency Minimization

Beyond auditing, the architecture developed here constitutes a formal mathematical model for **artificial metacognition**. The system does not merely produce outputs; it engages in second-order reasoning, evaluating its own first-order claims against structured constraints.

We propose the following thesis: > **Metacognition is the process of minimizing ℓ^1 inconsistency over a constraint graph, decomposed explicitly into repairable errors and irreducible domain ambiguities.**

The components of the ℓ^1 verification stack map directly onto the architecture of a metacognitive belief-revision framework:

Verification Architecture	Metacognitive Interpretation
Section (s)	The system's current **belief state**.
Coboundary ($\delta(s)$)	Detection of **internal contradiction**.
ℓ^1 norm (I)	Quantification of **cognitive dissonance**.
Repairable component (R)	**Correctable reasoning error** identified by the system.
Cohomological component (Φ)	**Irreducible uncertainty / ambiguity** inherent to the domain.
Autopoietic Gauge Iterator	The active **belief revision process** projecting to consistency.
Hallucination Certificate	A formal **self-assessment** outputting bounded confidence.

By separating errors mathematically, the verification layer converts an opaque predictive distribution into an agent capable of inspecting, diagnosing, and bounding its own epistemic failures, establishing a formal, falsifiable pipeline for reliable reasoning and Metacognition.

---

11. Implementation Path

11.1 Minimum Viable Product

[Layer B — engineering]

The smallest useful deployment of this framework requires:

1. **A domain graph** for one specific domain (e.g., drug interactions, tax law, building codes). 500–5,000 vertices, manually curated with domain expert input.
2. **A coboundary computer** that evaluates $I(s)$ given a section. This is $O(|E|)$ — trivial to implement.
3. **An LP solver** for the Hodge decomposition. Standard off-the-shelf (GLPK, HiGHS, Gurobi).
4. **A certificate generator** that produces the tuple $(I, \Phi, R, \eta, \mathcal{C})$ in a structured format.

This MVP provides Mode 1 (post-hoc audit) for a single domain. It can be built and deployed in weeks, not months.

11.2 Full Stack

The full ℓ^1 verification stack adds:

- **AGI repair loop** (Mode 2) — implements the autopoietic iterator from Paper 005.

- **Multi-domain composition** — categorical framework from Paper 006 for verifying across domain boundaries.
- **Adaptive refinement** — autopoietic subdivision for domain graph improvement.
- **Training integration** (Mode 3) — ℓ^1 coboundary loss for end-to-end training. [Layer C]

11.3 Mathematical Specification of Implementation Components

The full verification stack admits efficient implementation. The mathematical framework decomposes into the following computational components:

- **Coboundary auditor**: Computes $I(s) = \sum_e \|\delta_e(s)\|$ in $O(|E|)$ time by evaluating defects at each edge independently. Locality of the ℓ^1 norm ensures no global state is required.
- **Hodge decomposition solver**: Solves the LP $\min_{f \in C^0} \|\delta(s) - d_0 f\|_1$ to separate exact (repairable) from harmonic (irreducible) components. Standard network simplex or interior-point methods achieve polynomial complexity in the graph size.
- **Defect pattern classifier**: Analyzes the support of the defect field $\delta(s)$ to identify recurring structural patterns (e.g., cycles violating transitivity, conflicting paths, dimensional mismatches). This is a topological analysis of the coboundary's support structure, implementable via standard graph algorithms.
- **Epistemic filter**: Given a threshold τ, accepts outputs with $I(s) \leq \tau$ and rejects outputs with $I(s) > \tau$. For more granular control, applies thresholds to $R(s)$ (repairable defect) and Φ (irreducible floor) separately.
- **Certificate generator**: Constructs the tuple $(I, \Phi, R, \eta, \mathcal{C})$ by combining outputs from the coboundary auditor, Hodge solver, and cohomology computation. The certificate is a structured data object suitable for logging, auditing, or contract enforcement.

The mathematical framework guarantees that these components compose correctly: the output of the Hodge decomposition feeds the AGI repair loop, the repaired section feeds the coboundary auditor for verification, and the final defect score is provably bounded by Φ. No component requires model access or training data — all operate on the domain graph and output section.

---

12. Conclusion

The ℓ^1 coboundary norm is the unique defect measure compatible with local, additive, faithful observation. This is a theorem, not a design choice. Current

AI systems are trained under ℓ^2 objectives that permit cancellation, interpolation, and hidden inconsistency. The structural consequence is hallucination — confident outputs that correspond to no coherent worldview.

The framework presented here provides:

1. **A computable hallucination score** $I(s)$ for any AI output on any graph-modeled domain.
2. **A decomposition** into repairable errors (R) and irreducible domain complexity (Φ).
3. **A repair algorithm** (AGI) with proved monotone convergence and a cohomological floor.
4. **A certificate** $(I, \Phi, R, \eta, \mathcal{C})$ that can be audited, composed, and placed in contracts.
5. **A categorical framework** for transferring verification across related domains.

The mathematical foundations are proved (Papers 000–005). The mathematical framework admits efficient implementation. The engineering path from theorem to deployment is short: build a domain graph, compute the coboundary, solve the LP, issue the certificate.

AI systems equipped with ℓ^1 verification layers admit explicit, computable bounds on structural inconsistency.

Appendix: Background for ML Researchers

[Layer A — pedagogical exposition of standard mathematical concepts in ML-accessible terms]

This appendix presents the core mathematical concepts (presheaves, coboundaries, Hodge decomposition) in concrete graph-theoretic language, without category-theoretic formalism. The goal is to make the framework accessible to machine learning researchers with undergraduate linear algebra and graph theory.

A.1 Domain Graphs as Directed Graphs with Constraints

What is a domain graph?

A domain graph is just a finite directed graph $G = (V, E)$ where: - Each **vertex** $v \in V$ represents a concept or claim (e.g., "the patient has a fever", "temperature = 38.5°C"). - Each **edge** $e = (u, v) \in E$ represents a constraint or relationship (e.g., "if Flu then Fever", "temperature > 37°C implies fever").

Unlike standard graph embeddings, we don't learn continuous representations. Instead, we work with explicit **sections**: assignments of values to vertices.

What is a section?

A section s is a function $s : V \to F$ that assigns a value $s(v) \in F$ to each vertex. The fiber F is the value space — it could be: - $F = \mathbb{R}$ (real numbers) - $F = [0, 1]$ (probabilities) - $F = \{0, 1\}$ (binary labels) - $F = \mathbb{R}^k$ (vectors)

An AI system's output on a domain is a section: it assigns a value (claim, probability, label) to each concept in the graph.

What is a restriction map?

Each edge $e = (u, v)$ carries a **restriction map** $r_e : F \to F$ that says: "if the value at v is x, what does that imply for the value at u?"

Example: If edge e represents "if Flu (prob p) then Fever (prob $\geq 0.8p$)", the restriction map is $r_e(p) = 0.8p$.

A.2 The Coboundary: Measuring Inconsistency

Definition: The coboundary $\delta_e(s)$ at edge $e = (u, v)$ is:

$$\delta_e(s) = r_e(s(v)) - s(u)$$

This measures the **mismatch** between: - What vertex v claims (projected through the constraint e): $r_e(s(v))$ - What vertex u claims directly: $s(u)$

If $\delta_e(s) = 0$, the claims are consistent. If $\delta_e(s) \neq 0$, there's a contradiction.

The total defect is:

$$I(s) = \sum_{e \in E} |\delta_e(s)|$$

This is just the sum of absolute violations across all edges — the ℓ^1 norm of the coboundary vector.

Why ℓ^1 instead of ℓ^2?

The ℓ^2 norm $\sqrt{\sum_e \delta_e^2}$ permits cancellation: errors of opposite sign partially cancel. The ℓ^1 norm $\sum_e |\delta_e|$ does not. Every nonzero defect contributes to the total. This prevents averaging between contradictory claims.

A.3 The Hodge Decomposition: Repairable vs. Irreducible Defects

Question: Given a defect field $\delta = (\delta_{e_1}, \delta_{e_2}, \ldots, \delta_{e_m}) \in \mathbb{R}^m$ (one number per edge), how much of it can be fixed by adjusting the section s?

Answer: Solve the following **linear program**:

$$\Phi = \min_{f: V \to \mathbb{R}} \sum_{e=(u,v) \in E} |\delta_e - (f(v) - f(u))|$$

Here, f is a **gauge transformation** — a function that shifts the section values. The term $f(v) - f(u)$ is the coboundary of f at edge e, written $d_0 f(e)$.

Interpretation: - The LP finds the optimal shift f that brings δ as close as possible to the **exact form** $d_0 f$ (a coboundary of a 0-cochain). - The residual Φ is what remains after the best possible gauge shift. This is the **irreducible defect** — it cannot be removed by changing s alone.

Decomposition:

$$\delta = d_0 f^* + r^*$$

where: - $d_0 f^* = (f^*(v) - f^*(u))$ is the **exact (repairable) component** — fix this by adjusting $s(v) \leftarrow s(v) - f^*(v)$ for all v. - r^* is the **harmonic (irreducible) component** with $\|r^*\|_1 = \Phi$ — this cannot be removed without changing the domain graph.

Why this matters:

If $\Phi > 0$, the domain graph itself contains contradictions. Even a perfect AI cannot achieve $I = 0$ on this domain. Knowing Φ tells you where the domain model needs fixing.

A.4 The Autopoietic Repair Algorithm

The AGI (Autopoietic Gauge Iterator) repairs the section by iteratively applying a simple update rule:

For each vertex v with defect: 1. Collect all neighbors' projected values: $\{r_e(s(u)) : e = (u, v) \text{ or } e = (v, u)\}$ 2. Replace $s(v)$ with the ℓ^1 **median** of these projected values.

The **median** (not mean!) ensures no averaging. The update snaps to a value that minimizes the total ℓ^1 deviation from neighbors.

Convergence: Each iteration strictly decreases the total defect $I(s)$ until it reaches Φ (the irreducible floor). The algorithm stops when no further improvement is possible.

Complexity: For a graph with $|V|$ vertices and $|E|$ edges, the algorithm converges in $O(|V| \cdot I_0/\varepsilon)$ iterations in the worst case, where I_0 is the initial defect and ε is the tolerance. Under non-degeneracy, convergence is geometric: $O(\log(1/\varepsilon))$ iterations.

A.5 Linear Programming Formulation (Concrete)

For readers familiar with LP solvers, here is the explicit formulation for the Hodge decomposition:

Variables: $f_1, \ldots, f_n \in \mathbb{R}$ (one per vertex in $V = \{v_1, \ldots, v_n\}$)

Auxiliary variables: $z_1, \ldots, z_m \geq 0$ (one per edge in E)

Objective:

$$\min \sum_{j=1}^{m} z_j$$

Constraints: For each edge $e_j = (v_i, v_k)$:

$$-z_j \leq \delta_{e_j} - (f_k - f_i) \leq z_j$$

This is a standard LP with $n + m$ variables and $2m$ inequality constraints. Any off-the-shelf LP solver (GLPK, HiGHS, Gurobi, CPLEX) can solve it in polynomial time.

Output: The optimal $f^* = (f_1^*, \ldots, f_n^*)$ and the minimal cost $\Phi = \sum_j z_j^*$.

A.6 Comparison with Familiar ML Concepts

ML Concept	ℓ^1 Framework Analog
Feature vector	Section s (assigns values to vertices)
Loss function	Coboundary defect $I(s) = \sum_e \lVert\delta_e\rVert$
Training data	Domain graph G (constraints, not samples)
Regularization	Irreducible defect Φ (structural lower bound)
Gradient descent	AGI median updates (iterative repair)
Validation error	Total defect $I(s)$ on test graph
Overfitting	Section fits one subgraph but violates global constraints

Key difference: Standard ML minimizes loss over training samples. The ℓ^1 framework minimizes defect over **structural constraints**. It is not statistical — it is topological.

A.7 When to Use This Framework

Use the ℓ^1 coboundary framework when: - The domain has explicit constraints (logical, causal, ontological). - Inconsistency has consequences (medical, legal, financial, safety-critical). - You need a **certificate** of consistency, not just a probability. - The deployment domain differs from the training domain (out-of-distribution).

Do not use this framework when: - The domain has no explicit constraints (purely perceptual tasks like image recognition). - Statistical uncertainty is the primary concern (use conformal prediction). - You only care about average-case performance (use standard ℓ^2 metrics).

A.8 Minimal Working Example (Python Pseudocode)

```python
import numpy as np
from scipy.optimize import linprog

# Define domain graph
V = ['Fever', 'Headache', 'Flu', 'Migraine', 'Dehydration']
E = [('Flu', 'Fever', 0.8), ('Flu', 'Headache', 0.6), ...]  # (u, v, weight)

# AI output (section)
s = {'Fever': 0.7, 'Headache': 0.8, 'Flu': 0.6, 'Migraine': 0.5, 'Dehydration': 0.4}

# Compute coboundary defects
defects = []
for (u, v, w) in E:
    delta_e = w * s[v] - s[u]  # simplified restriction map: r_e(x) = w*x
    defects.append(abs(delta_e))

# Total defect
I = sum(defects)
print(f"Total defect: {I}")

# Hodge decomposition (LP formulation)
# Variables: f[v] for each vertex v
# Minimize: sum |delta_e - (f[v] - f[u])|
c = np.ones(len(E))  # objective: minimize sum of z_j
# ... construct A_ub, b_ub from constraints -z_j <= delta_e - (f_v - f_u) <= z_j
# res = linprog(c, A_ub, b_ub)
# Phi = res.fun

# AGI repair (iterative median updates)
for iteration in range(max_iters):
    for v in V:
        neighbors = [...]  # collect projected values from neighbors
        s[v] = np.median(neighbors)  # L1 median update
    I_new = compute_total_defect(s)
    if I_new >= I - epsilon:
        break  # converged
    I = I_new

# Certificate
certificate = (I, Phi, I - Phi, (I - Phi) / I)
print(f"Certificate: {certificate}")
```

This pseudo code captures the entire pipeline: domain graph construction, coboundary computation, Hodge decomposition via LP, AGI repair, and certifi-

cate generation.

References

[1] J. H. Carroll, "Projection Obstruction Theory: Retraction Nonlinearity, ℓ^1 Rigidity, and Density Scaling," 2026. DOI: 10.5281/zenodo.19151803

[2] J. H. Carroll, "The Free ℓ^1 Seminorm on Banach Presheaf Coboundaries," 2026. DOI: 10.5281/zenodo.19152115

[3] J. H. Carroll, "Coordinate-Wise Additivity and the ℓ^1 Norm on Finite Graph Cochains," 2026. DOI: 10.5281/zenodo.19152599

[4] J. H. Carroll, "Hodge Structure and Gauge Equivalence of ℓ^1 Defect Fields," 2026. DOI: 10.5281/zenodo.19152799

[5] J. H. Carroll, "Autopoietic Cohomology: Iterative Obstruction Repair, Manifold Emergence, and Gauge Structure on Causal Posets," 2026. DOI: 10.5281/zenodo.19155011

[6] J. H. Carroll, "A Common ℓ^1 Defect Framework for Quantum, Causal, and Gravitational Systems," 2026. DOI: 10.5281/zenodo.19155079

[7] J. H. Carroll, "Constraint Closure and the ℓ^1 Structure of Physical Law," 2026. DOI: 10.5281/zenodo.19183020

[8] A. Chambolle, "An Algorithm for Total Variation Minimization and Applications," *J. Math. Imaging Vis.* 20, 89–97, 2004.

[9] E. J. Candes and T. Tao, "Decoding by Linear Programming," *IEEE Trans. Inf. Theory* 51(12), 4203–4215, 2005.

[10] L. Ambrosio, N. Gigli, and G. Savare, *Gradient Flows in Metric Spaces and in the Space of Probability Measures*, Birkhauser, 2008.

[11] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[12] Y. Bengio, A. Courville, and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Trans. PAMI* 35(8), 2013.